

Rapport d'exécution des tests web

Généré par TestFlow - Moteur intelligent de génération et d'exécution de tests

URL testée : <https://practicetestautomation.com/contact/>

Date de génération : 2026-06-11 20:37:21

1. Résumé exécutif

L'exécution des tests est globalement réussie. Tous les cas de test exécutés ont été validés avec succès. Aucune anomalie bloquante n'a été détectée pendant l'exécution automatisée.

2. Informations générales de l'analyse

Élément	Valeur
ID analyse	194
URL testée	https://practicetestautomation.com/contact/
Type de page détecté	Formulaire de contact
Fonctionnalité principale	contact form
Scope d'analyse	Fonctionnalité précise
Types de tests générés	Tests fonctionnels, Tests UI, Tests de sécurité, Tests SEO
Version	
Date d'exécution	2026-06-11 19:15:39.344731

3. Statistiques globales

3
Total

3
PASS

0
FAIL

0
PENDING

Statut	Nombre	Pourcentage
PASS	3	100.0%
FAIL	0	0.0%
PENDING	0	0.0%

4. Détail complet des cas de test

1. TC_CONTACT_004_InvalidEmailFormat

ID : 2624

PASS

Type : negative Priorité : high Version : v5

Étapes du test

1. Naviguer vers la page de contact.
2. Saisir 'John' dans le champ 'First Name'.
3. Saisir 'Doe' dans le champ 'Last Name'.
4. Saisir 'invalid-email' dans le champ 'Email' (ID: wpforms-161-field_1).
5. Saisir 'Ceci est un message avec un email invalide.' dans le champ 'Comment or Message'.
6. Cliquer sur le bouton 'Submit'.

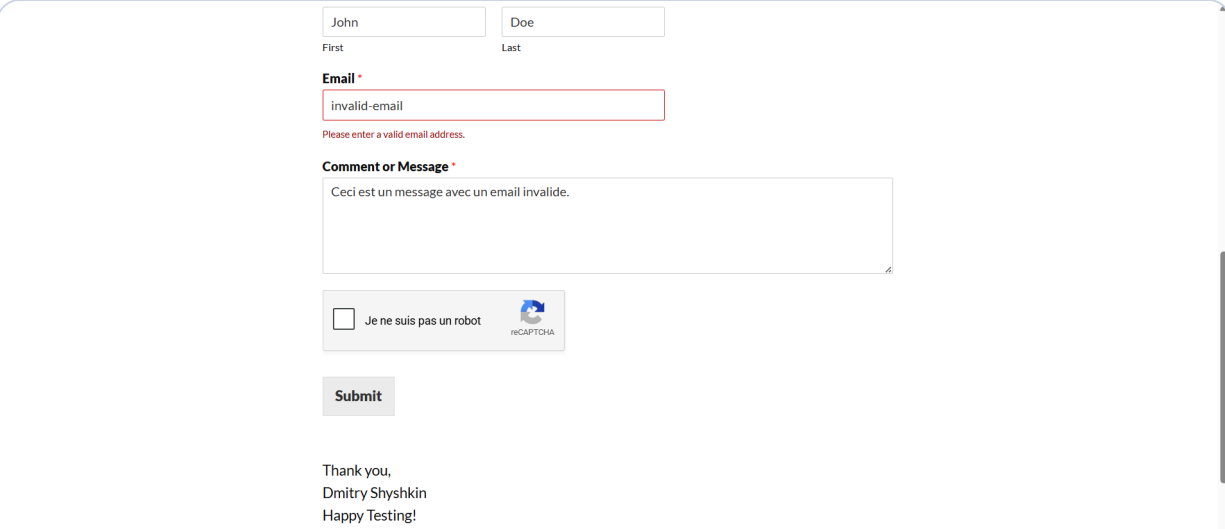
Résultat attendu

Le formulaire n'est pas soumis. Un message d'erreur de validation HTML5 est affiché pour le champ 'Email', indiquant un format invalide.

Résultat réel / message d'exécution

Test négatif validé. Message d'erreur détecté : invalid

Capture d'écran



The screenshot shows a contact form with the following elements:

- First Name field: John
- Last Name field: Doe
- Email field: invalid-email (highlighted with a red border and a red error message: "Please enter a valid email address.")
- Comment or Message field: Ceci est un message avec un email invalide.
- Recaptcha: A checkbox labeled "Je ne suis pas un robot" with a recaptcha logo.
- Submit button: A button labeled "Submit".
- Footer: "Thank you, Dmitry Shyshkin Happy Testing!"

▼ Voir le script Selenium

```
driver.get("https://practicetestautomation.com/contact/")

first_name_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_0"), (By.NAME, "first_name")])
first_name_input.send_keys("John")
```

```
last_name_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_0-last"), (By.NAME, "last_name_input")])
last_name_input.send_keys("Doe")

email_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_1"), (By.NAME, "email_input")])
email_input.send_keys("invalid-email")

comment_textarea = safe_find_element(driver, [(By.ID, "wpforms-161-field_2"), (By.NAME, "comment_textarea")])
comment_textarea.send_keys("Ceci est un message avec un email invalide.")

submit_button = safe_find_element(driver, [(By.ID, "wpforms-submit-161"), (By.NAME, "submit_button")])
submit_button.click()

assert "Thank you, Dmitry Shyshkin Happy Testing!" not in driver.page_source, "Le message de remerciement est toujours présent"
```

▼ Voir le script Cypress

```
cy.visit('https://practicetestautomation.com/contact/')
cy.get('#wpforms-161-field_0').type('John')
cy.get('#wpforms-161-field_0-last').type('Doe')
cy.get('#wpforms-161-field_1').type('invalid-email')
cy.get('#wpforms-161-field_2').type('Ceci est un message avec un email invalide.')
cy.get('#wpforms-submit-161').click()
cy.contains('Thank you, Dmitry Shyshkin Happy Testing!').should('not.exist')
```

2. TC_CONTACT_005_MissingRequiredField_Comment

ID : 2625

PASS

Type : negative Priorité : medium Version : v5

Étapes du test

1. Naviguer vers la page de contact.
2. Saisir 'John' dans le champ 'First Name' (ID: wpforms-161-field_0).
3. Saisir 'Doe' dans le champ 'Last Name' (ID: wpforms-161-field_0-last).
4. Saisir 'john.doe@example.com' dans le champ 'Email' (ID: wpforms-161-field_1).
5. Laisser le champ 'Comment or Message' (ID: wpforms-161-field_2) vide.
6. Cliquer sur le bouton 'Submit' (ID: wpforms-submit-161).

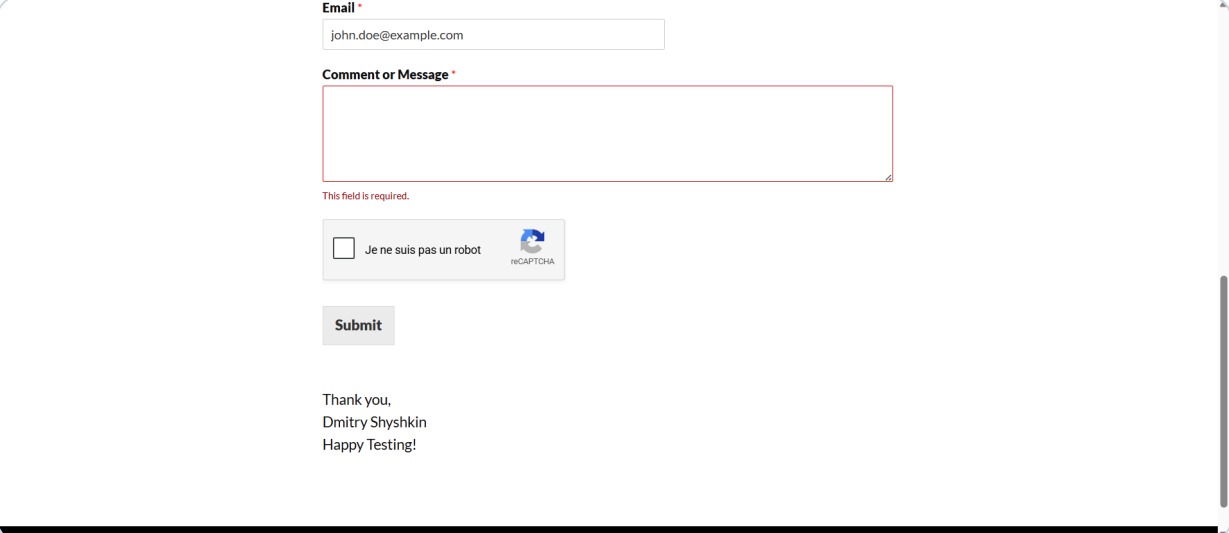
Résultat attendu

Le formulaire n'est pas soumis. Un message d'erreur de validation HTML5 est affiché pour le champ 'Comment or Message', indiquant qu'il s'agit d'un champ obligatoire.

Résultat réel / message d'exécution

Test négatif validé. Message d'erreur détecté : invalid

Capture d'écran



The screenshot shows a web form with the following elements:

- Email ***: A text input field containing 'john.doe@example.com'.
- Comment or Message ***: A large text area that is empty. Below it, a red error message states: 'This field is required.'
- reCAPTCHA**: A checkbox labeled 'Je ne suis pas un robot' with a reCAPTCHA logo.
- Submit**: A button to submit the form.
- Footer**: Text at the bottom reads 'Thank you, Dmitry Shyshkin Happy Testing!'.

▼ Voir le script Selenium

```
driver.get("https://practicetestautomation.com/contact/")

first_name_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_0"), (By.NAME, "first_name")])
first_name_input.send_keys("John")

last_name_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_0-last"), (By.NAME, "last_name")])
last_name_input.send_keys("Doe")

email_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_1"), (By.NAME, "email")])
email_input.send_keys("john.doe@example.com")

comment_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_2"), (By.NAME, "comment")])
comment_input.send_keys("")

submit_button = safe_find_element(driver, [(By.ID, "wpforms-submit-161"), (By.NAME, "submit")])
submit_button.click()
```

```
last_name_input.send_keys("Doe")

email_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_1"), (By.NAME, "
email_input.send_keys("john.doe@example.com")

# Leave comment textarea empty

submit_button = safe_find_element(driver, [(By.ID, "wpforms-submit-161"), (By.NAME,
submit_button.click()

assert "Thank you, Dmitry Shyshkin Happy Testing!" not in driver.page_source, "Le me
```

▼ Voir le script Cypress

```
cy.visit('https://practicetestautomation.com/contact/')
cy.get('#wpforms-161-field_0').type('John')
cy.get('#wpforms-161-field_0-last').type('Doe')
cy.get('#wpforms-161-field_1').type('john.doe@example.com')
// Leave comment field empty
cy.get('#wpforms-submit-161').click()
cy.contains('Thank you, Dmitry Shyshkin Happy Testing!').should('not.exist')
```

3. TC_CONTACT_006_XSSAttempt_CommentField

ID : 2626

PASS

Type : security Priorité : medium Version : v5

Étapes du test

1. Naviguer vers la page de contact.
2. Saisir 'John' dans le champ 'First Name'.
3. Saisir 'Doe' dans le champ 'Last Name'.
4. Saisir 'john.doe@example.com' dans le champ 'Email'.
5. Saisir '<script>alert('XSS');</script>' dans le champ 'Comment or Message' (ID: wpforms-161-field_2).
6. Cliquer sur le bouton 'Submit'.

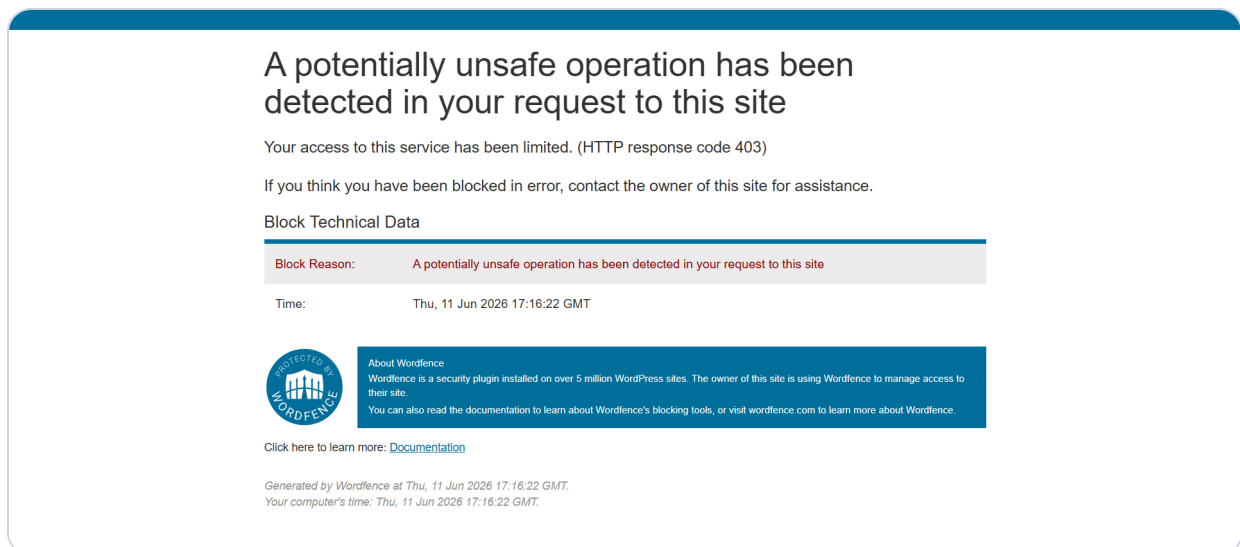
Résultat attendu

Le script XSS ne doit pas s'exécuter. Idéalement, le contenu soumis est encodé ou filtré par le serveur. Aucune pop-up d'alerte ne doit apparaître.

Résultat réel / message d'exécution

Test XSS sécurisé : aucune alerte JavaScript détectée. La requête dangereuse a été bloquée par une protection du site.

Capture d'écran



▼ Voir le script Selenium

```
driver.get("https://practicetestautomation.com/contact/")

first_name_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_0"), (By.NAME, "first_name")])
```

```

first_name_input.send_keys("John")

last_name_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_0-last"), (By.ID, "wpforms-161-field_0-last")], (By.ID, "wpforms-161-field_0-last"))
last_name_input.send_keys("Doe")

email_input = safe_find_element(driver, [(By.ID, "wpforms-161-field_1"), (By.NAME, "wpforms-161-field_1")], (By.ID, "wpforms-161-field_1"))
email_input.send_keys("john.doe@example.com")

comment_textarea = safe_find_element(driver, [(By.ID, "wpforms-161-field_2"), (By.NAME, "wpforms-161-field_2")], (By.ID, "wpforms-161-field_2"))
comment_textarea.send_keys("<script>alert('XSS');</script>")

submit_button = safe_find_element(driver, [(By.ID, "wpforms-submit-161"), (By.NAME, "wpforms-submit-161")], (By.ID, "wpforms-submit-161"))
submit_button.click()

# Check if an alert was triggered (Selenium can catch JS alerts)
from selenium.common.exceptions import NoAlertPresentException

try:
    driver.switch_to.alert.accept()
    assert False, "A XSS alert was triggered, indicating a vulnerability."
except NoAlertPresentException:
    assert True, "No XSS alert was triggered."

# Also check if the 'Thank you' message appears, indicating normal submission (after successful submission)
assert "Thank you, Dmitry Shyshkin Happy Testing!" in driver.page_source, "Le formulaire n'a pas été soumis correctement."

```

▼ Voir le script Cypress

```

cy.visit('https://practicetestautomation.com/contact/')
cy.get('#wpforms-161-field_0').type('John')
cy.get('#wpforms-161-field_0-last').type('Doe')
cy.get('#wpforms-161-field_1').type('john.doe@example.com')
cy.get('#wpforms-161-field_2').type('<script>alert(\'XSS\');</script>')

// Stub window.alert to prevent it from blocking the test
cy.window().then((win) => {
    cy.stub(win, 'alert').as('alertStub');
});

cy.get('#wpforms-submit-161').click()

// Assert that the alert was not called
cy.get('@alertStub').should('not.have.been.called')
// Check for successful submission message to ensure the form processed the input (happy path)
cy.contains('Thank you, Dmitry Shyshkin Happy Testing!').should('be.visible')

```

5. Tests échoués

Aucun test dans cette catégorie.

6. Tests en attente / PENDING

7. Recommandations

- Continuer à exécuter ces tests après chaque modification de la page afin d'éviter les régressions.
- Conserver les captures d'écran comme preuve visuelle de l'exécution.
- Mettre à jour les scripts de test si la structure HTML de la page change.

8. Conclusion

Ce rapport fournit une vision détaillée de l'exécution des tests générés automatiquement. Il peut être utilisé par l'équipe de développement pour identifier les anomalies, vérifier les comportements attendus et prioriser les corrections nécessaires.